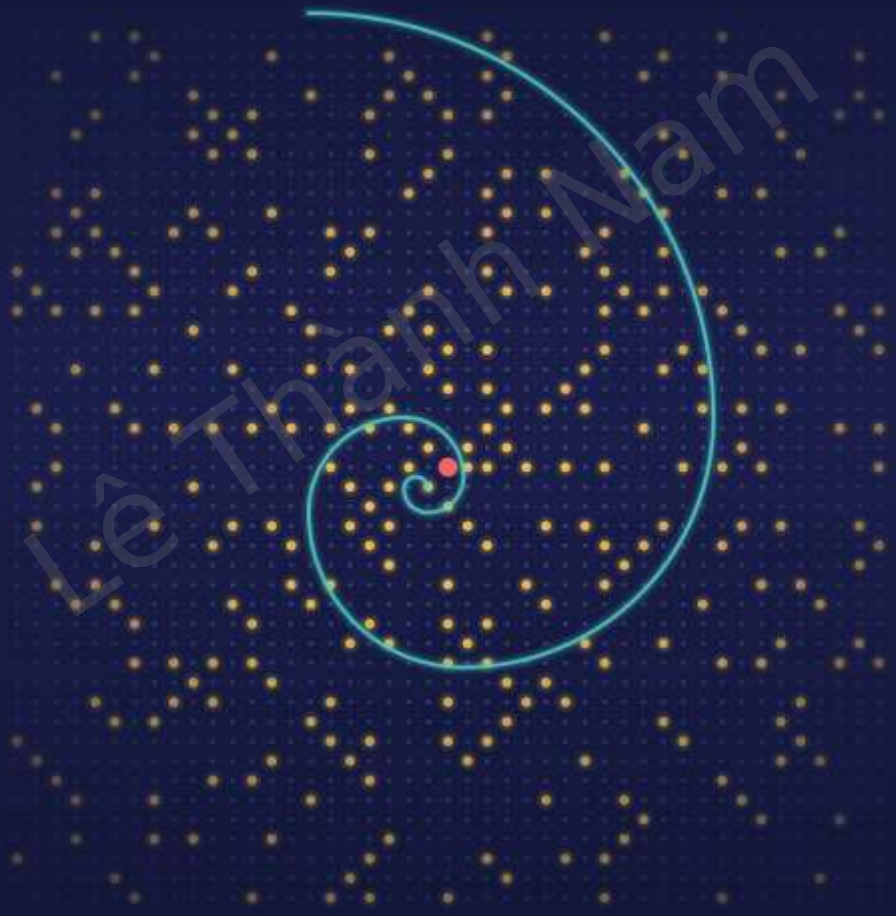
CHUYÊN ĐỀ BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

TOÁN HỌC TRONG TIN HỌC

qua 36 bài Timus · từ cơ bản đến nâng cao



Xoắn ốc Ulam: số nguyên tố (vàng) xếp thành những đường chéo bí ẩn; đường xanh: xoắn ốc Fibonacci

9 chương · 36 bài · 104 chương trình đã kiểm chứng và đo thời gian thật

Số nguyên tố · sàng | Hệ cơ số | Đồng dư · Euclid | Hàm số học | Tổ hợp

Số nguyên lớn | Dãy số · truy hồi | Lý thuyết trò chơi | Hình học tính toán

LÊ THÀNH NAM

Lê Thành Nam

MỤC LỤC

Giới thiệu **3**

Chương 1. Số nguyên tố và sàng Eratosthenes **6**

Bài 1. Mật mã (số nguyên tố thứ n) (Timus 1086) 7
Bài 2. Những số "không tầm thường" (Timus 1118) 12
Bài 3. Một bài dễ hơn (Goldbach) (Timus 1356) 17
Bài 4. Michael và mật mã học (Timus 2102) 22

Chương 2. Hệ cơ số, chữ số và tính tuần hoàn **26**

Bài 5. Đừng hỏi tuổi phụ nữ (Timus 1104) 27
Bài 6. Kẻ tốt, kẻ xấu và kẻ xấu xí (Timus 2138) 32
Bài 7. 1, 10, 100, 1000… (Timus 1209) 36
Bài 8. Ý tưởng điên rồ (Timus 1295) 40

Chương 3. Đồng dư, Euclid mở rộng và lũy thừa mô-đun **44**

Bài 9. Tội ác và hình phạt (Timus 1430) 45
Bài 10. Phần tử lũy đẳng (Timus 1204) 49
Bài 11. Tấn công RSA (Timus 1141) 54
Bài 12. Căn bậc hai theo mô-đun (Timus 1132) 58

Chương 4. Ước số và các hàm số học **62**

Bài 13. Những nhà khinh khí cầu dũng cảm (Timus 1049) 63
Bài 14. Lại chuyện cái đầu hói (Timus 1355) 66
Bài 15. Số phức tạp nhất (Timus 1748) 69
Bài 16. Những số thú vị (Timus 2070) 73

Chương 5. Tổ hợp và đếm **77**

Bài 17. Hoán vị (Timus 1024) 78
Bài 18. Tổ hợp (Timus 1055) 82
Bài 19. Kì thi ở Tmutarakan (Timus 1091) 86
Bài 20. Những tấm vé "xui xẻo" (Timus 1217) 90

Chương 6. Số nguyên lớn **95**

Bài 21. Đại bàng Chernobyl (Timus 1222) 96
Bài 22. Tổng chữ số của tổng hai số (Timus 1206) 100
Bài 23. Số thú vị (Timus 1385) 104
Bài 24. Trò chơi bốc sỏi (Timus 1180) 108

Chương 7. Dãy số và đại số tuyến tính **112**

Bài 25. Dãy Fibonacci (Timus 1133) 113
Bài 26. Những phép tính đơn giản (Timus 1047) 117
Bài 27. Dãy số của quỷ (Timus 1605) 121

Chương 8. Lý thuyết trò chơi **125**

Bài 28. Cúc áo (Timus 1023) 126
Bài 29. Đến lúc lấy sỏi (Timus 1087) 130
Bài 30. Trò chơi hạt dẻ (Timus 2068) 134
Bài 31. Trò chơi vui (Timus 1045) 138

Chương 9. Hình học tính toán **142**

Bài 32. Sợi dây (Timus 1020) 143
Bài 33. Con dê trong vườn (Timus 1084) 146

Lê Thanh Nam

Bài 34. Quả bóng trong mơ (Timus 1192) 150
Bài 35. Bức tường (Timus 1185) 153
Bài 36. Trung vị trên mặt phẳng (Timus 1207) 158

Lê Thành Nam

GIỚI THIỆU

Cuốn sách này dành cho ai

Sách gồm 36 bài **toán học trong tin học** chọn từ kho đề Timus (acm.timus.ru) của Đại học Liên bang Ural (Nga) – một trong những kho đề lâu đời và "khó tính" nhất thế giới. Các bài được xếp thành 9 chương theo **dạng toán**: số nguyên tố và sàng; hệ cơ số, chữ số và chu kì; đồng dư, Euclid mở rộng và lũy thừa mô-đun; ước số và hàm số học; tổ hợp và đếm; số nguyên lớn; dãy số và đại số; lý thuyết trò chơi; hình học tính toán.

Đề Timus phần lớn lấy từ các kì thi học sinh, sinh viên của Nga (vòng loại ICPC khu vực Ural, NEERC, các cuộc thi của trường Đại học Ural). Đặc trưng của chúng là đề ngắn, ràng buộc chặt, và **cái khó nằm ở toán**: một quan sát đúng biến bài toán "không thể" thành vài dòng code. Cuốn sách này giúp học sinh rèn đúng kĩ năng đó: thử với số nhỏ, đoán quy luật, chứng minh, rồi cài đặt cẩn thận.

Mỗi bài gồm hai phần. **Đề bài** được viết lại bằng tiếng Việt, giữ đúng yêu cầu, ràng buộc, định dạng vào/ra và ví dụ của đề gốc, kèm **bảng subtask** do sách bổ sung. **Hướng dẫn giải** đi qua nhiều **tầng**: lời giải ngây thơ, các tầng cải tiến (kể cả những tầng "gần đúng" mắc bẫy tràn số, sai số thực, thiếu trường hợp), rồi mới đến lời giải tối ưu. Mỗi tầng đều có code chạy được, thời gian chạy đo thật và hình minh hoạ.

> **Về cách viết đề**
> *Đề trong sách không phải bản dịch từng câu. Mỗi đề được viết lại theo cách hiểu tự nhiên của người Việt, bỏ những chi tiết kể chuyện không ảnh hưởng đến lời giải. Mọi điều kiện, giới hạn, định dạng vào/ra và test ví dụ được giữ nguyên; mọi ví dụ đã được đối chiếu bằng cách chạy lời giải. Giáo viên nên mở đề gốc theo đường dẫn in trong từng bài để đối chiếu chi tiết. Giới hạn thời gian ghi trong sách là giới hạn của Timus.*

CÁCH GIẢI

Hướng dẫn giải của mỗi bài chia thành các phần có dải màu riêng. Trong mỗi phần, lời giải đi từng bước nhỏ, kèm hình minh hoạ và chứng minh ngắn gọn cho mọi khẳng định toán học.

Phần	Nội dung
HIỂU ĐỀ	Diễn đạt lại bài toán bằng ngôn ngữ toán học; tính tay ví dụ; vẽ hình
NGÂY THƠ	Tầng 1: làm đúng như đề nói (vét cạn, mô phỏng, thử mọi khả năng), code và độ phức tạp
BẾ TẮC	Thời gian chạy đo thật của tầng 1, chỉ ra nguyên nhân chậm, kết thúc bằng một câu hỏi then chốt
CẢI TIẾN 1, 2, …	Các tầng trung gian: mỗi tầng sửa đúng một điểm yếu của tầng trước; có cả những tầng "nhanh nhưng sai" (tràn số, sai số thực, nhầm điều kiện, quên trường hợp đặc biệt) để học sinh thấy lỗi thường gặp
LÓE SÁNG	Tính chất toán học quyết định dẫn tới lời giải tối ưu, có chứng minh và hình minh hoạ
TỐI ƯU	Code tối ưu, độ phức tạp và bảng so sánh tất cả các tầng trên mọi subtask
LƯU Ý	Những chỗ dễ sai và phần mở rộng

Thời gian chạy trong sách được đo thật bằng trình biên dịch g++ (tuỳ chọn -O2, chuẩn C++17) trên máy chủ Linux, tính cả thời gian đọc và ghi dữ liệu. Máy đo của sách và máy chấm Timus khác nhau, nên các con số tuyệt đối chỉ mang tính tham khảo; tỉ lệ giữa các tầng mới là điều quan trọng.

SUBTASK VÀ CÁC TẦNG LỜI GIẢI

Timus chấm theo kiểu "tất cả hoặc không": chỉ cần sai một test là bài không được chấp nhận. Để học sinh thấy rõ giá trị của từng ý tưởng, sách bổ sung **bảng subtask** cho mỗi bài: subtask đầu dành cho lời giải ngây thơ, các subtask giữa dành cho các tầng cải tiến, subtask cuối là ràng buộc gốc. Với những bài mà ràng buộc gốc nhỏ tới mức một lời giải trung bình cũng qua được, sách **mở rộng** ràng buộc ở subtask cuối để lời giải tối ưu thật sự cần thiết; các subtask mở rộng được ghi rõ. Test lớn của mỗi subtask được dựng có chủ đích để "bẫy" đúng điểm yếu của từng tầng (số nguyên tố lớn, khoảng trống giữa các số nguyên tố, tích làm tràn 64 bit, …).

Cuối mỗi bài có bảng so sánh: mỗi tầng chạy trên test lớn nhất của từng subtask, ghi thời gian thật và số điểm đạt được. Ô đỏ nghĩa là quá giờ, sai kết quả hoặc lỗi khi chạy.

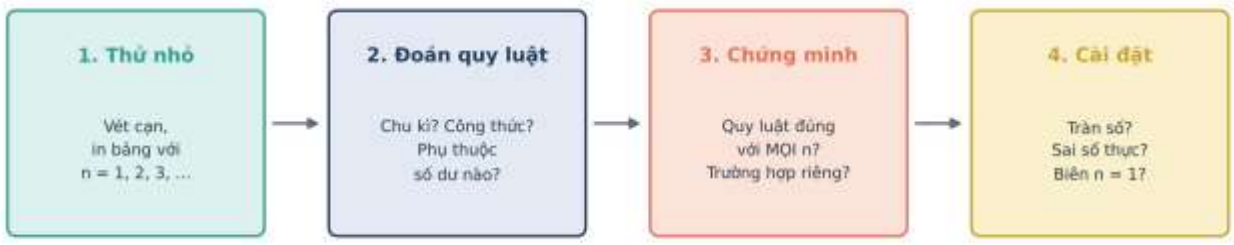
SỬ DỤNG SÁCH

Với mỗi bài, học sinh nên đọc đề, **tự tính tay với số nhỏ**, tự nghĩ ít nhất 20–30 phút rồi mới đọc hướng dẫn, và dừng lại ở mỗi "Câu hỏi then chốt" để tự trả lời trước khi đọc tiếp. Mọi code trong sách đã được chạy với ví dụ của đề và đối chiếu chéo với nhau trên 120 test ngẫu nhiên mỗi bài (trừ những tầng được cố ý trình bày như một lời giải sai). Những bài có nhiều đáp án đúng được chấm bằng chương trình kiểm tra; những bài in số thực được so sánh với sai số cho phép.

Nộp bài trên Timus

- Đăng kí tài khoản tại acm.timus.ru (mục Register). Timus gửi cho bạn một **mã JUDGE_ID** – mã này thay cho mật khẩu khi nộp bài.
- Mở đề theo số bài (ví dụ 1086) bằng đường dẫn trong sách, bấm Submit, nhập JUDGE_ID, chọn ngôn ngữ (ví dụ G++ 13 C++20), dán code và nộp.
- Chương trình đọc từ bàn phím (stdin) và ghi ra màn hình (stdout).
- Kết quả: Accepted (đúng), Wrong answer (sai đáp án), Time limit exceeded (quá thời gian), Memory limit exceeded (quá bộ nhớ), Runtime error (lỗi khi chạy). Timus báo số hiệu test đầu tiên bị sai – một gợi ý quý giá.

04 bước tư duy trước một bài toán học



Hình 36.3. Bốn bước xây dựng lời giải một bài toán học trong tin học

- **Bước 1. Thử nhỏ.** Viết vét cạn, in bảng đáp án với n = 1, 2, 3, … Phần lớn quy luật hiện ra ngay trên bảng này.

Lê Thành Nam

- **Bước 2. Đoán quy luật.** Chu kì? Công thức? Chỉ phụ thuộc vào số dư nào? Đáp án có phải số nguyên tố, lũy thừa, tổ hợp?
- **Bước 3. Chứng minh.** Một quy luật chưa chứng minh có thể sai ở test thứ 37. Chứng minh còn chỉ ra các trường hợp đặc biệt cần xử lí riêng.
- **Bước 4. Cài đặt cẩn thận.** Tràn số (dùng long long, __int128 hay số lớn?), sai số thực (long double, làm tròn), giới hạn bộ nhớ của sàng, các trường hợp biên (n = 1, 0, số âm).

Dạng toán	Công cụ chính	Độ phức tạp điển hình
Kiểm tra / liệt kê số nguyên tố	thử chia tới √n; sàng Eratosthenes	O(√n); O(n log log n)
Phân tích thừa số	thử chia bởi các số nguyên tố ≤ √n	O(√n / ln n)
Ước chung, phương trình ax + by = c	Euclid, Euclid mở rộng	O(log n)
Lũy thừa, nghịch đảo mô-đun	lũy thừa nhanh, định lý Fermat / Euler	O(log n)
Hệ đồng dư	định lý Trung Hoa về số dư	O(log n)
Hàm số học (số ước, tổng ước)	phân tích thừa số; sàng	O(√n); O(n log n)
Đếm tổ hợp	nhân – cộng, bao hàm – loại trừ, Legendre	tuỳ bài
Đáp án rất lớn	số nguyên lớn (mảng chữ số cơ số 10^9)	O(độ dài²)
Trò chơi hai người	thắng/thua bằng quy hoạch động, Sprague–Grundy, bất biến	tuỳ bài
Hình học	tích có hướng, bao lồi, công thức diện tích	O(n log n)

Bố cục sách

Chương	Chủ đề	Bài (mã Timus)
1	Số nguyên tố và sàng Eratosthenes	1086, 1118, 1356, 2102
2	Hệ cơ số, chữ số và tính tuần hoàn	1104, 2138, 1209, 1295
3	Đồng dư, Euclid mở rộng và lũy thừa mô-đun	1430, 1204, 1141, 1132
4	Ước số và các hàm số học	1049, 1355, 1748, 2070
5	Tổ hợp và đếm	1024, 1055, 1091, 1217
6	Số nguyên lớn	1222, 1206, 1385, 1180
7	Dãy số và đại số tuyến tính	1133, 1047, 1605
8	Lý thuyết trò chơi	1023, 1087, 2068, 1045
9	Hình học tính toán	1020, 1084, 1192, 1185, 1207

Lê Thành Nam

Chương 1. Số nguyên tố và sàng Eratosthenes

> **Trọng tâm chương**
> Số nguyên tố là "nguyên tử" của số học. Chương này luyện ba kĩ năng nền: **thử chia tới căn bậc hai**, **sàng Eratosthenes** (khi cần nhiều số nguyên tố) và dùng **định lý về phân bố số nguyên tố** (Bertrand, Goldbach, khoảng cách giữa các số nguyên tố) để biến bài toán tìm kiếm thành vài phép kiểm tra.

Kiến thức cần nhớ

- n là hợp số ⇒ n có ước nguyên tố ≤ √n: thử chia tới √n là đủ.
- Sàng Eratosthenes: gạch các bội của mỗi số nguyên tố từ p^2 – O(n log log n).
- Định lý Bertrand: với n ≥ 2 luôn có số nguyên tố p, n < p < 2n.
- Giả thuyết Goldbach (đã kiểm tra tới $4 \cdot 10^{18}$): mọi số chẵn > 2 là tổng của hai số nguyên tố.
- Dưới 10^9, khoảng cách giữa hai số nguyên tố liên tiếp không quá 282.

Các bài trong chương

Bài	Tên bài	Mã Timus	Nguồn	Số tầng
1	Mật mã (số nguyên tố thứ n)	1086	Cuộc thi lập trình học sinh lần 3, ĐH Ural, 2001	3
2	Những số "không tầm thường"	1118	USU Open Collegiate Contest, 10/2001, Junior	3
3	Một bài dễ hơn (Goldbach)	1356	Giải vô địch trẻ ĐH Ural, 3/2005	3
4	Michael và mật mã học	2102	Giải vô địch trẻ ĐH Liên bang Ural, 2016	3

Bài 1. Mật mã (số nguyên tố thứ n)

Mã bài Timus: 1086

Nguồn: Cuộc thi lập trình học sinh lần 3, ĐH Ural, 2001 · Giới hạn thời gian sách dùng: 2,0 giây · Bộ nhớ: 64 MB
Đề gốc: https://acm.timus.ru/problem.aspx?space=1&num=1086

Đề bài

Ban giám khảo cần một chương trình tìm **số nguyên tố thứ n** để làm khoá cho hệ mật mã của họ. Một số nguyên dương là số nguyên tố nếu nó có đúng hai ước dương phân biệt (vì thế 1 không phải số nguyên tố). Các số nguyên tố theo thứ tự là 2, 3, 5, 7, 11, …

Cho k số n, với mỗi số hãy in số nguyên tố thứ n.

Dữ liệu vào

Dòng đầu chứa số nguyên dương k. Tiếp theo là k số nguyên dương n, mỗi số trên một dòng.

Kết quả

Với mỗi n, in số nguyên tố thứ n trên một dòng.

Ràng buộc

- $1 \le n \le 15\,000$.
- Đề gốc không nêu giới hạn của k; sách dùng $k \le 15\,000$.

Ví dụ

Dữ liệu vào	Kết quả
4 3 2 5 7	5 3 11 17

Giải thích ví dụ

Các số nguyên tố đầu tiên: 2, 3, 5, 7, 11, 13, 17. Số thứ 3 là 5, số thứ 2 là 3, số thứ 5 là 11, số thứ 7 là 17.

Subtask

Các subtask dưới đây do sách bổ sung để luyện từng mức thuật toán; subtask cuối cùng là ràng buộc của đề gốc (hoặc mở rộng hơn đề gốc). Giới hạn thời gian 2,0 giây cho mọi subtask.

Subtask	Ràng buộc	Điểm
ST1	$n \le 500$, $k \le 10$	30%
ST2	$n \le 15\,000$, $k \le 10$	30%
ST3	$n \le 15\,000$, $k \le 15\,000$	40%

Hướng dẫn giải

HIỂU ĐỀ Hiểu đúng đề bài

Bài hỏi phần tử thứ n của dãy số nguyên tố. Câu hỏi đầu tiên luôn là: **số nguyên tố thứ 15 000 lớn cỡ nào?** Theo định lý số nguyên tố, số nguyên tố thứ n xấp xỉ $n \cdot \ln n$; với n = 15 000 là khoảng $1{,}6 \cdot 10^5$. Chạy thử cho kết quả chính xác: $p_{15000} = 163\,841$. Vậy mọi số ta cần đều nhỏ hơn 170 000.

Cái khó của bài nằm ở **số truy vấn**: có thể có tới hàng nghìn truy vấn, mỗi truy vấn hỏi một n lớn.

NGÂY THƠ Ý tưởng ngây thơ: đi lần lượt và kiểm tra từng số

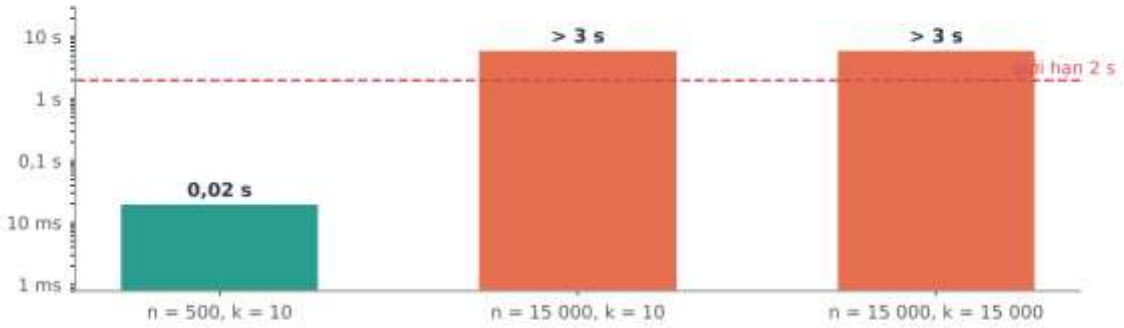
Đi m = 2, 3, 4, … và đếm số nguyên tố gặp được cho tới khi đủ n. Kiểm tra m nguyên tố bằng cách thử chia cho mọi d từ 2 tới m − 1.

Code. Tầng 1 – thử chia cho mọi d

```
#include <bits/stdc++.h>
using namespace std;
// Tầng 1: thử chia m cho mọi d từ 2 tới m - 1
bool isPrime(int m) {
    if (m < 2) return false;
    for (int d = 2; d < m; d++)
        if (m % d == 0) return false;
    return true;
}
int main() {
    int k; scanf("%d", &k);
    while (k--) {
        int n; scanf("%d", &n);
        int cnt = 0, m = 1;
        while (cnt < n) { m++; if (isPrime(m)) cnt++; }
        printf("%d\n", m);
    }
}
```

Để đến được số nguyên tố thứ n ta duyệt mọi $m \le p_n$, mỗi m tốn tới m phép chia: tổng cỡ $p_n^2/2 \approx 1{,}3 \cdot 10^{10}$ phép chia cho một truy vấn n = 15 000 (hợp số thì dừng sớm, nhưng các số nguyên tố thì phải đi hết).

BẾ TẮC Bế tắc: mỗi số nguyên tố tốn m phép chia



Hình 1.1. *Tầng 1 trên ba nhóm test*

Với $n \le 500$ ($p_{500} = 3571$) tầng 1 vẫn nhanh. Nhưng chỉ cần một truy vấn n = 15 000 là đã quá giờ.

Lê Thành Nam

Câu hỏi then chốt

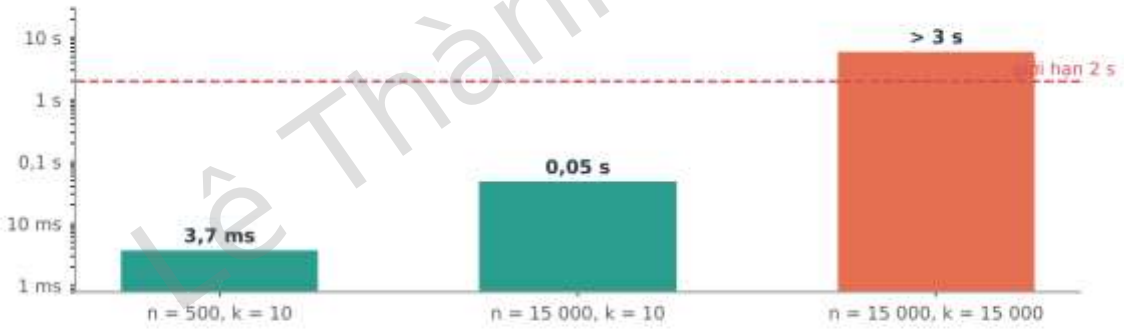
Muốn biết m có phải số nguyên tố, có thật cần thử chia cho mọi d < m?

CẢI TIẾN 1 Cải tiến 1: chỉ thử chia tới căn bậc hai

Nếu m là hợp số, m = a·b với 2 ≤ a ≤ b, thì $a^2 \le a \cdot b = m$, tức **m luôn có một ước a ≤ √m**. Vậy chỉ cần thử d với d·d ≤ m. Với m ≈ $1{,}6 \cdot 10^5$, mỗi lần kiểm tra chỉ còn tối đa 400 phép chia.

Code. Tầng 2 – thử chia tới √m

```
#include <bits/stdc++.h>
using namespace std;
// Tầng 2: chỉ thử chia tới căn bậc hai của m
bool isPrime(int m) {
    if (m < 2) return false;
    for (int d = 2; d * d <= m; d++)
        if (m % d == 0) return false;
    return true;
}
int main() {
    int k; scanf("%d", &k);
    while (k--) {
        int n; scanf("%d", &n);
        int cnt = 0, m = 1;
        while (cnt < n) { m++; if (isPrime(m)) cnt++; }
        printf("%d\n", m);
    }
}
```



Hình 1.2. Tầng 2 trên ba nhóm test

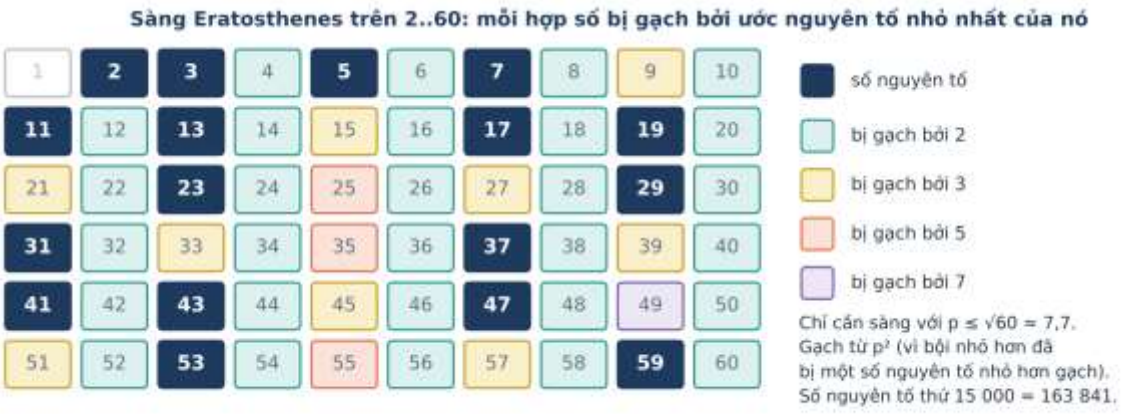
Một truy vấn n = 15 000 giờ chỉ mất vài chục mili-giây. Nhưng với 15 000 truy vấn, ta **làm đi làm lại cùng một việc**: mỗi truy vấn lại kiểm tra từ m = 2.

LÓE SÁNG Lóe sáng: tính tất cả một lần bằng sàng

Mọi truy vấn đều hỏi về cùng một dãy số nguyên tố dưới 170 000. Ta liệt kê **toàn bộ** dãy này một lần, lưu vào mảng pr, rồi trả lời truy vấn n bằng pr[n − 1].

Cách liệt kê nhanh nhất là **sàng Eratosthenes**: viết các số 2..M; lấy số nhỏ nhất chưa bị gạch (đó là số nguyên tố p), gạch mọi bội của p bắt đầu từ p^2 (các bội k·p với k < p đã bị gạch bởi một số nguyên tố nhỏ hơn); lặp lại.

Lê Thành Nam



Hình 1.3. Sàng Eratosthenes trên các số 2..60

Số phép gạch là Σ M/p trên các số nguyên tố p ≤ √M, xấp xỉ M·ln ln M – gần như tuyến tính.

TỐI ƯU Lời giải tối ưu

Code. Tầng 3 – sàng Eratosthenes và tra bảng

```
#include <bits/stdc++.h>
using namespace std;
// Tầng 3: sàng Eratosthenes một lần, trả lời mỗi truy vấn O(1)
const int LIM = 170000;          // số nguyên tố thứ 15000 là 163841
int main() {
    vector<char> comp(LIM + 1, 0);
    vector<int> pr;
    for (int i = 2; i <= LIM; i++) {
        if (comp[i]) continue;
        pr.push_back(i);
        for (long long j = 1LL * i * i; j <= LIM; j += i) comp[j] = 1;
    }
    int k; scanf("%d", &k);
    while (k--) {
        int n; scanf("%d", &n);
        printf("%d\n", pr[n - 1]);
    }
}
```

Lê Thành Nam

Sàng tới M = 170 000 mất chưa tới 1 mili-giây; mỗi truy vấn O(1). Tổng O(M log log M + k).

Tầng	Ý tưởng	Độ phức tạp	n = 500, k = 10 (ST1)	n = 15 000, k = 10 (ST2)	n = 15 000, k = 15 000 (ST3)	Điểm
1	Thử chia cho mọi d < m	$O(p_n^2)$ mỗi truy vấn	0,02 s	quá giờ	quá giờ	30
2	Thử chia tới √m	$O(p_n \cdot \sqrt{p_n})$ mỗi truy vấn	3,7 ms	0,05 s	quá giờ	60
3	Sàng một lần, tra bảng	O(M log log M + k)	3,7 ms	3,4 ms	6,4 ms	100

Thời gian đo thật trên test lớn nhất của từng subtask; giới hạn 2,0 giây. Ô đỏ: quá giờ, sai kết quả hoặc lỗi khi chạy (tràn bộ nhớ, tràn ngăn xếp). Cột "Điểm" cộng phần trăm của các subtask mà tầng đó vượt qua.

LƯU Ý Những chỗ dễ sai

- Mảng pr đánh số từ 0: số nguyên tố thứ n là pr[n − 1].

- Khi sàng, viết j = i·i bằng long long nếu M lớn (với M ≈ 10^5, i·i có thể vượt 2^{31} khi i ≈ 46 341 nếu không dừng vòng ngoài sớm).
- Phải chọn M đủ lớn: kiểm tra pr.size() ≥ 15 000. Nếu không biết trước, dùng ước lượng $p_n < n(\ln n + \ln \ln n)$ với n ≥ 6.
- Mẫu "nhiều truy vấn trên cùng một dãy" ⇒ tiền xử lý một lần là kĩ thuật sẽ gặp lại ở nhiều chương sau.

Lê Thành Nam